

ENTERPRISE PLATFORM

Low-Level Design Document

LLD — PART 1 OF 2

Database Schema | API Contracts | Service Interfaces | Data Models

Document Scope

This is Part 1 of the 2-part LLD. The HLD defined what the system does and how it is structured at a high level. This LLD defines exactly how Module 1 (Identity and Authentication) is implemented at the code level. Every table, every column, every API endpoint, every request and response shape is defined here with enough detail that a developer can implement from this document alone.

Field	Detail
Module	Module 1 — Identity and Authentication
Scope	Database schema, API contracts, service interfaces, data models
Stack	FastAPI + SQLAlchemy + asyncpg + PostgreSQL + Redis
Related HLD	HLD Parts 1, 2, and 3
LLD Part 2 covers	Sequence diagrams, error catalogue, data flow, test strategy

1. Database Schema — auth Schema

All Module 1 tables live in the auth PostgreSQL schema. No other module reads or writes these tables directly. The schema is created by the first Alembic migration and all changes are version controlled.

1.1 Table: auth.users

The central identity table. Every user in the system has exactly one row here. All other modules reference auth.users.id as a foreign key.

Column	Type	Nullable	Description
id	UUID	NOT NULL	Primary key. Generated as uuid4. Used by all modules as user reference.
tenant_id	UUID	NOT NULL	Multi-tenancy identifier. Default: platform tenant UUID. Row-level security uses this.
username	VARCHAR(30)	NOT NULL	Unique per tenant. Pattern: <code>^[a-zA-Z0-9_!\-]+\</code>
email	VARCHAR(254)	NOT NULL	Unique per tenant. Stored lowercase. Validated via email-validator library.
phone	VARCHAR(20)	NOT NULL	E.164 format. e.g. +919876543210. Validated via libphonenumber.
full_name	VARCHAR(150)	NOT NULL	Display name. Not used for authentication.
hashed_password	VARCHAR(128)	NULL	bcrypt hash. NULL for Google OAuth-only accounts.
google_sub	VARCHAR(128)	NULL	Google OAuth subject identifier. Unique globally.
email_verified	BOOLEAN	NOT NULL	Default FALSE. Login blocked until TRUE.
phone_verified	BOOLEAN	NOT NULL	Default FALSE. Login blocked until TRUE.
is_active	BOOLEAN	NOT NULL	Default TRUE. FALSE = soft-deleted or banned.
is_superuser	BOOLEAN	NOT NULL	Default FALSE. Platform operator access only.
created_at	TIMESTAMPTZ	NOT NULL	UTC. Set on insert. Never updated.
updated_at	TIMESTAMPTZ	NOT NULL	UTC. Auto-updated on any row change.
last_login_at	TIMESTAMPTZ	NULL	UTC. Updated on every successful login.

Indexes on auth.users:

UNIQUE INDEX uq_users_email ON auth.users (email)

```

UNIQUE INDEX uq_users_phone    ON auth.users (phone)
UNIQUE INDEX uq_users_username ON auth.users (username)
UNIQUE INDEX uq_users_google  ON auth.users (google_sub)
INDEX ix_users_email          ON auth.users (email)
INDEX ix_users_phone          ON auth.users (phone)
INDEX ix_users_username       ON auth.users (username)
INDEX ix_users_tenant         ON auth.users (tenant_id)

```

1.2 Table: auth.user_sessions

One row per issued refresh token. Refresh token rotation creates a new row and marks the old one invalid. Never deleted — provides complete session history.

Column	Type	Nullable	Description
id	UUID	NOT NULL	Primary key.
user_id	UUID	NOT NULL	FK → auth.users.id ON DELETE CASCADE.
token_hash	VARCHAR(64)	NOT NULL	SHA-256 hex digest of the raw refresh token. Raw token never stored.
is_valid	BOOLEAN	NOT NULL	Default TRUE. Set FALSE on logout, rotation, or reuse detection.
user_agent	VARCHAR(512)	NULL	Browser or client user-agent string. Truncated at 512 chars.
ip_address	VARCHAR(45)	NULL	IPv4 or IPv6. 45 chars covers full IPv6 representation.
created_at	TIMESTAMPTZ	NOT NULL	When this session was created.
expires_at	TIMESTAMPTZ	NOT NULL	When the refresh token expires. Used to clean up old rows.
last_used_at	TIMESTAMPTZ	NULL	Updated on every successful token refresh.

Indexes on auth.user_sessions:

```

UNIQUE INDEX uq_sessions_token ON auth.user_sessions (token_hash)
INDEX ix_sessions_user_id      ON auth.user_sessions (user_id)
INDEX ix_sessions_token_hash   ON auth.user_sessions (token_hash)

```

```

-- Migration 0002 adds these performance indexes:
PARTIAL INDEX idx_sessions_user_valid
ON auth.user_sessions (user_id)
WHERE is_valid = true
CONCURRENTLY

```

1.3 Table: auth.audit_logs

Immutable append-only security event log. No UPDATE or DELETE ever runs on this table. Provides complete history of every security-relevant action.

Column	Type	Nullable	Description
id	BIGSERIAL	NOT NULL	Auto-incrementing integer. Efficient for time-ordered inserts.
user_id	UUID	NULL	FK → auth.users.id ON DELETE SET NULL. NULL for pre-registration events.
tenant_id	UUID	NULL	For multi-tenant audit filtering.
event	VARCHAR(64)	NOT NULL	Event type. See full event catalogue below.
ip_address	VARCHAR(45)	NULL	Client IP address.
user_agent	VARCHAR(512)	NULL	Client user-agent string.
detail	JSONB	NULL	Extra context. e.g. {method: 'email_otp', country: 'India'}
created_at	TIMESTAMPTZ	NOT NULL	UTC. Set on insert. Indexed for time-range queries.

Indexes on auth.audit_logs:

```
INDEX ix_audit_user_id    ON auth.audit_logs (user_id)
INDEX ix_audit_event      ON auth.audit_logs (event)
INDEX ix_audit_created_at ON auth.audit_logs (created_at)

-- Migration 0002 adds:
COMPOSITE INDEX idx_audit_logs_user_created
  ON auth.audit_logs (user_id, created_at DESC)
  CONCURRENTLY

-- GIN index for JSONB detail queries:
GIN INDEX idx_audit_detail ON auth.audit_logs USING gin (detail)
```

Complete audit event catalogue:

Event Value	When It Is Written
register	User successfully completes registration
register_google	User created via Google OAuth for first time
email_verified	User's email address confirmed
phone_verified	User's phone number confirmed
login_success	Any successful login by any method
login_failed_password	Wrong password submitted

Event Value	When It Is Written
logout	User explicitly logs out
password_reset_requested	Reset email requested
password_reset_completed	Password successfully changed
refresh_token_reuse_detected	Old refresh token presented — all sessions killed

2. API Contracts — All Endpoints

Every endpoint is defined with its full request schema, response schema, possible error codes, rate limit, and authentication requirement. This is the definitive contract between the backend and all clients.

2.1 POST /api/v1/auth/register

Property	Detail
Method	POST
Auth required	No
Rate limit	3 requests per hour per IP
Success response	HTTP 201 Created
Side effects	Sends verification email + SMS OTP. Creates user record. Writes audit log.

Request body:

```
{
  "email": "user@example.com", // required, normalised to lowercase
  "phone": "+919876543210",    // required, E.164 format
  "username": "john_doe",      // required, 3-30 chars, alphanumeric+_-
  "password": "StrongPass@123", // required, min 10 chars, complexity rules
  "full_name": "John Doe"      // required, 1-150 chars
}
```

Success response (201):

```
{
  "data": {
    "user_id": "uuid",
    "message": "Account created. Please verify email and phone.",
    "email_verified": false,
    "phone_verified": false
  }
}
```

Error responses:

409 DUPLICATE_FIELD — email, phone, or username already registered
422 VALIDATION_ERROR — input fails schema validation
429 RATE_LIMIT — too many registration attempts

2.2 POST /api/v1/auth/verify/email

Method: POST
Auth: No

Limit: No specific limit (token is single-use)

Request body:

```
{
  "token": "base64url-encoded string" // from email link or manual entry
}
```

Success response (200):

```
{
  "data": {
    "verified": true,
    "message": "Email verified successfully."
  }
}
```

Error responses:

400 INVALID_TOKEN — token invalid, expired, or already used
404 USER_NOT_FOUND — no user matches token

2.3 POST /api/v1/auth/verify/phone

Method: POST

Auth: No

Limit: 5 requests per 15 minutes per IP

Request body:

```
{
  "phone": "+919876543210", // E.164 format
  "otp": "123456" // exactly 6 digits
}
```

Success response (200):

```
{
  "data": {
    "verified": true,
    "message": "Phone number verified successfully."
  }
}
```

Error responses:

400 INVALID_OTP — OTP wrong, expired, or max attempts exceeded
404 USER_NOT_FOUND
429 RATE_LIMIT

2.4 POST /api/v1/auth/login/password

Method: POST

Auth: No

Limit: 10 requests per 10 minutes per IP

Request body:

```
{
  "username_or_email": "john_doe or user@example.com",
  "password": "StrongPass@123"
}
```

Success response (200):

```
{
  "data": {
    "user_id": "uuid",
    "username": "john_doe",
    "email": "user@example.com",
    "full_name": "John Doe",
    "is_active": true,
    "email_verified": true,
    "phone_verified": true,
    "token_metadata": {
      "access_token_expires_at": "2024-01-16T15:00:00Z",
      "refresh_token_expires_at": "2024-01-23T14:45:00Z",
      "token_type": "Bearer"
    }
  }
}
```

Cookies set (HttpOnly + Secure + SameSite=Strict):

access_token — JWT, 15 min, path=/
refresh_token — JWT, 7 days, path=/api/v1/auth/refresh

Error responses:

401 INVALID_CREDENTIALS — wrong username/password
403 ACCOUNT_NOT_VERIFIED — email or phone not yet verified
403 ACCOUNT_INACTIVE — account deactivated
429 RATE_LIMIT

2.5 POST /api/v1/auth/login/otp/email/request

Method: POST

Auth: No

Limit: 5 requests per 15 minutes per IP

Request body:

```
{
  "email": "user@example.com"
}
```

Response (202 — always, regardless of whether email exists):

```
{
  "data": {
    "message": "If this email is registered, a login code has been sent."
  }
}
```


Note: 202 Accepted is intentional. Never 404 for unknown email.
This prevents email enumeration attacks.

2.6 POST /api/v1/auth/login/otp/email/verify

Method: POST

Auth: No

Limit: 10 requests per 10 minutes per IP

Request body:

```
{
  "identifier": "user@example.com", // email address
  "otp":       "123456"             // 6 digits
}
```

Success response (200): same shape as login/password success

Cookies set: same as login/password

Error responses:

401 INVALID_OTP — wrong OTP, expired, or already used

403 ACCOUNT_NOT_VERIFIED

403 ACCOUNT_INACTIVE

429 RATE_LIMIT

2.7 POST /api/v1/auth/login/otp/sms/request

Method: POST

Auth: No

Limit: 5 requests per 15 minutes per IP

Request body:

```
{
  "phone": "+919876543210" // E.164 format
}
```

Response (202 — always): same anti-enumeration pattern as email OTP

2.8 POST /api/v1/auth/login/otp/sms/verify

Method: POST

Auth: No

Limit: 10 requests per 10 minutes per IP

Request body:

```
{
  "identifier": "+919876543210", // E.164 phone
  "otp":       "123456"         // 6 digits
}
```

Success response (200): same shape as login/password success

2.9 GET /api/v1/auth/login/google

Method: GET
Auth: No
Limit: No specific limit

No request body.

Response (200):

```
{
  "data": {
    "auth_url": "https://accounts.google.com/o/oauth2/v2/auth?..."
  }
}
```

Side effects:
Sets oauth_state cookie (HttpOnly, 5 min TTL)
Frontend redirects browser to auth_url

2.10 GET /api/v1/auth/google/callback

Method: GET
Auth: No (handled internally via OAuth state)
Limit: No specific limit

Query parameters:
code — authorization code from Google
state — CSRF state token (validated against oauth_state cookie)

Success response (200): same shape as login/password success
Cookies set: same as login/password

Error responses:
400 INVALID_STATE — state mismatch, possible CSRF attempt
403 ACCOUNT_INACTIVE

2.11 POST /api/v1/auth/password-reset/request

Method: POST
Auth: No
Limit: 3 requests per hour per IP (reuses register_rate_limit)

Request body:
{
 "email": "user@example.com"
}

Response (200 — always):
{
 "data": {
 "message": "If this email is registered, a reset link has been sent."
 }
}

Note: Always 200. Never reveals if email is registered.

2.12 POST /api/v1/auth/password-reset/confirm

Method: POST
Auth: No
Limit: No specific limit (token is single-use, expires in 30 min)

Request body:
{
 "token": "raw-reset-token-from-email",
 "new_password": "NewStrongPass@456"
}

Success response (200):
{
 "data": {
 "message": "Password reset. All sessions signed out."
 }
}

Error responses:
400 INVALID_TOKEN — token invalid, expired, or already used
400 ACCOUNT_INACTIVE
422 VALIDATION_ERROR — new_password fails complexity rules

2.13 POST /api/v1/auth/refresh

Method: POST
Auth: Via refresh_token HttpOnly cookie
Limit: No specific limit

No request body.

Success response (200):

```
{
  "data": {
    "access_token_expires_at": "2024-01-16T15:00:00Z",
    "message": "Tokens rotated successfully."
  }
}
```

Cookies set: new access_token + new refresh_token
Old cookies: overwritten

Error responses:

- 401 NO_TOKEN — refresh_token cookie missing
- 401 INVALID_TOKEN — token invalid or expired
- 401 SESSION_INVALID — reuse detected, all sessions killed
- 401 ACCOUNT_INACTIVE

2.14 POST /api/v1/auth/logout

Method: POST

Auth: Via refresh_token HttpOnly cookie (optional — logs out even without it)

Limit: No specific limit

No request body.

Success response (200):

```
{
  "data": {
    "message": "Logged out successfully."
  }
}
```

Side effects:

- Marks session as is_valid=false in database
- Clears access_token and refresh_token cookies
- Writes logout event to audit_logs

2.15 GET /api/v1/auth/me

Method: GET

Auth: Via access_token HttpOnly cookie — required

Limit: No specific limit

No request body.

Success response (200): same shape as login/password success

Error responses:

- 401 NOT_AUTHENTICATED — access_token cookie missing
- 401 INVALID_TOKEN — token invalid or expired
- 404 USER_NOT_FOUND — user deleted after token issued

3. Service Interfaces — Internal Contracts

These are the internal Python interfaces that other modules in the Modular Monolith use to interact with Module 1. In Phase 3 when modules are extracted, these interfaces become HTTP API calls or gRPC contracts.

3.1 Identity Service Interface

```
# app/services/interfaces/identity.py
# This is the contract other modules use to interact with auth

from dataclasses import dataclass
from datetime import datetime

@dataclass
class UserIdentity:
    """Minimal identity object returned to other modules."""
    id: str
    email: str
    username: str
    full_name: str
    is_active: bool
    email_verified: bool
    phone_verified: bool
    tenant_id: str
    created_at: datetime

class IdentityServiceInterface:
    """
    Interface contract for Module 1.
    Other modules import and use this — never the ORM model.
    In Phase 3 this becomes an HTTP client to the auth microservice.
    """

    async def get_user_by_id(
        self,
        user_id: str,
    ) -> UserIdentity | None:
        """Returns user identity or None if not found."""
        ...

    async def get_user_by_email(
        self,
        email: str,
        tenant_id: str,
    ) -> UserIdentity | None:
        """Returns user identity or None. Tenant-scoped."""
        ...

    async def validate_token(
```

```

    self,
    access_token: str,
) -> UserIdentity | None:
    """Validates JWT and returns user identity or None."""
    ...

async def user_has_permission(
    self,
    user_id: str,
    permission: str,
) -> bool:
    """
    Checks if user has the given permission.
    Permission format: module:action
    Examples: orders:write, admin:dashboard
    """
    ...

```

3.2 Token Validation Dependency

The shared dependency that any endpoint in any module can use to get the current authenticated user:

```

# app/core/dependencies.py
# Shared across all modules — in shared kernel

from typing import Annotated
from fastapi import Cookie, Depends, HTTPException, status
from jose import JWTError
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.security import decode_access_token
from app.db.session import get_db
from app.models.user import User
from sqlalchemy import select

async def get_current_user(
    db: Annotated[AsyncSession, Depends(get_db)],
    access_token: Annotated[str | None, Cookie()] = None,
) -> User:
    """
    FastAPI dependency — returns authenticated User.
    Raises HTTP 401 if not authenticated.

    Usage in any module endpoint:
    @router.get('/profiles/{id}')
    async def get_profile(
        user: Annotated[User, Depends(get_current_user)],
    ):
        ...
    """

```

```

if not access_token:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail={'code': 'NOT_AUTHENTICATED',
                'message': 'Not authenticated.'},
    )
try:
    payload = decode_access_token(access_token)
except JWTErrors:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail={'code': 'INVALID_TOKEN',
                'message': 'Invalid or expired token.'},
    )
result = await db.execute(
    select(User).where(User.id == payload['sub'])
)
user = result.scalar_one_or_none()
if not user or not user.is_active:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail={'code': 'USER_NOT_FOUND',
                'message': 'User not found.'},
    )
return user

```

```

# Convenience alias used in endpoint signatures
CurrentUser = Annotated[User, Depends(get_current_user)]

```

3.3 Permission Decorator

RBAC permission enforcement for future modules. Defined in the shared kernel now so all modules use the same pattern:

```

# app/core/permissions.py

from functools import wraps
from fastapi import HTTPException, status
from app.models.user import User

def require_permission(permission: str):
    """
    Decorator for endpoints requiring specific permissions.

    Usage:
    @router.post('/orders')
    @require_permission('orders:write')
    async def create_order(
        user: CurrentUser,
        ...
    )
    """

```



```
):  
...
```

Permission format: {module}:{action}

Defined permissions:

```
auth:read_any    — read any user record  
auth:write_any   — modify any user record  
profiles:write   — update profile data  
orders:read      — read order data  
orders:write     — create or update orders  
admin:dashboard  — access admin dashboard  
admin:users      — manage user accounts  
...
```

```
def decorator(func):  
    @wraps(func)  
    async def wrapper(*args, user: User, **kwargs):  
        if not _user_has_permission(user, permission):  
            raise HTTPException(  
                status_code=status.HTTP_403_FORBIDDEN,  
                detail={  
                    'code': 'PERMISSION_DENIED',  
                    'message': f'Permission required: {permission}',  
                },  
            )  
        return await func(*args, user=user, **kwargs)  
    return wrapper  
return decorator
```

```
def _user_has_permission(user: User, permission: str) -> bool:  
    """  
    Phase 1: simple role-based check.  
    Phase 2: replaced by database lookup against  
        admin.permissions table.  
    """  
    if user.is_superuser:  
        return True  
    # Expand with full RBAC in Module 5  
    return False
```

4. Pydantic Data Models — Complete Validation Rules

Every input field has explicit validation rules. These are enforced by Pydantic v2 before any business logic runs. Invalid data never reaches the database.

Field	Type	Validation Rules
email	EmailStr	Valid email format. Normalised to lowercase. Max 254 chars (RFC 5321).
phone	str	Must parse as valid E.164 via libphonenumber. Stored in E.164 format.
username	str	3-30 chars. Pattern: <code>^[a-zA-Z0-9_.\-]+\$</code> No spaces. No special chars except <code>. _ -</code>
password	SecretStr	Min 10 chars. Must contain: 1 uppercase, 1 lowercase, 1 digit, 1 special char.
full_name	str	1-150 chars. Whitespace stripped. Any Unicode characters allowed.
otp	str	Exactly 6 digits. Whitespace stripped. Pattern: <code>^\d{6}\$</code>
token	str	Min 10 chars. Max 512 chars. Opaque string — no format validation.
identifier	str	Email or E.164 phone. Validated based on format detected.

5. Redis Data Models — Complete Key Specifications

Key Pattern	TTL	Value Structure
<code>otp:{channel}:{identifier}</code>	600s	Hash: {otp: '123456', signature: 'hex', attempts: '0'}
<code>pwd_reset:{sha256_hash}</code>	1800s	String: user_id UUID
<code>rl:{endpoint}:{ip}</code>	window_seconds	Sorted set: {timestamp_ms: score}
<code>session_cache:{user_id}</code>	900s	JSON string: serialised UserIdentity object
<code>oauth_state:{state_value}</code>	300s	String: '1' (existence check only)

LLD Part 1 Complete

Part 2 covers: Complete sequence diagrams for all major flows (registration, OTP login, token refresh, password reset), the error handling catalogue, data flow diagrams, security implementation details, and the complete test strategy with test cases for each endpoint. Reply 'LLD Part 2' to continue.